

HEPHAESTUS

Auditoría de contrato, ejecución completa y resultados medidos del runtime de inferencia C++17 para el decodificador nano de PROMETHEUS-NS: kernels SIMD propios sin dependencias, verificación numérica contra el oráculo NumPy, tokenizer BPE byte-level con paridad HuggingFace, cuantización progresiva int8/int4/ternaria, KV-cache int8 paginada y benchmark honesto contra el baseline PyTorch.

VERIFICACIÓN NUMÉRICA

Greedy exacto vs oráculo · max diff 4.8e-07 · coseno 0.99999991

CUANTIZACIÓN MEDIDA

int8 1.64x tokens/s con ppl +0.04% · ternaria +29.8% reportada

SUITE DE VERIFICACIÓN

49 tests en verde (42 estructura + 2 ctest + 5 golden) · 12 commits

RESTRICCIÓN DE CONTRATO

C++17 CPU-only · cero dependencias externas · AVX-512 detectado en runtime

Sucesor directo de PROMETHEUS-NS

Stack de IA engineering: del entrenamiento CPU-first al runtime de inferencia optimizado con cuantización extrema

Índice

1. Resumen ejecutivo	1
2. Arquitectura del sistema	2
3. Verificación numérica: el oráculo como verdad	2
4. Kernels propios: SIMD sin dependencias	3
4.1. El bug #1: BPE byte-level en C++	4
5. Cuantización progresiva: resultados medidos	4
6. KV-cache int8 paginada	5
7. Condicionales del paquete → precauciones implementadas	5
8. Auditoría de contrato y trazabilidad	6
9. Límites honestos y trabajo futuro	7
10. Conclusiones	7
A. CLI del motor (contrato del Makefile)	7
B. Historial de commits	7
C. Reproducibilidad	7
D. Artefactos entregados	9

1 Resumen ejecutivo

HEPHAESTUS es el sucesor directo de PROMETHEUS-NS en el stack de IA engineering: un **runtime de inferencia en C++17, 100 % CPU y con cero dependencias externas** (sin BLAS, sin Eigen, sin nlohmann) que carga los pesos safetensors del modelo nano entrenado por PROMETHEUS-NS, soporta *exactamente* esa arquitectura (6 capas pre-norm, RMSNorm, SwiGLU, RoPE half-split, GQA con $n_{kv} = n_h = 6$, embeddings atados, vocabulario 8000) y mide rendimiento real contra el baseline PyTorch. La restricción de arquitectura fija no es una limitación: es la premisa que permite optimizar a fondo cada kernel.

El proyecto se rige por una regla de honestidad estricta: toda cifra de tokens/s, RAM o perplejidad proviene de un benchmark ejecutado cuyo log queda commiteado; prohibido estimar speedups sin medirlos. En consecuencia, este informe no contiene un solo número proyectado: las secciones 5 y 6 reportan mediciones ejecutadas en el contenedor de referencia (2 vCPU x86-64 con AVX-512 disponible, compilación `-O3 -march=native`), y los hallazgos desfavorables se documentan con el mismo detalle que los favorables.

Resultado principal

El motor reproduce al oráculo NumPy (la especificación numérica) con greedy **exacto** en 64 tokens por prompt y logits con **max-abs-diff** 4.8×10^{-7} (tolerancia contractual 10^{-2} : margen de 20 810×) y **coseno mínimo 0.99999991** sobre el vocabulario completo. En cuantización, **int8 gana 1.64× tokens/s con una degradación de perplejidad de +0.04 %**; int4 cumple el umbral contractual (+0.89 % < 5 %) y la cuantización ternaria degrada +29.8 % — el hallazgo esperado y reportado sin maquillaje para un modelo no entrenado con QAT.

La ejecución cubrió las siete fases del contrato (0–6) con su verificación explícita: esqueleto con tests de estructura en verde, loader safetensors propio con digest SHA-256 por tensor, forward fp32 verificado contra el oráculo, generación greedy con determinismo exacto, cuantización progresiva medible y reversible, KV-cache int8 paginada con máscara de página parcial, y suite de benchmark con reporte autogenerado. El árbol final contiene 12 commits convencionales, 49 archivos fuente y 49 tests en verde (42 de estructura + 2 de ctest + 5 golden/pytest).

2 Arquitectura del sistema

La figura 1 resume el flujo completo. La frontera Python→C++ queda reducida a artefactos de datos verificables: safetensors + tabla de tensores + manifest + tokenizer exportado. El motor nunca enlaza bibliotecas de terceros; el oráculo NumPy (`tests/golden/reference_model.py`) es la especificación numérica viva y los golden tests comparan ambos mundos en cada ejecución de `make test`.

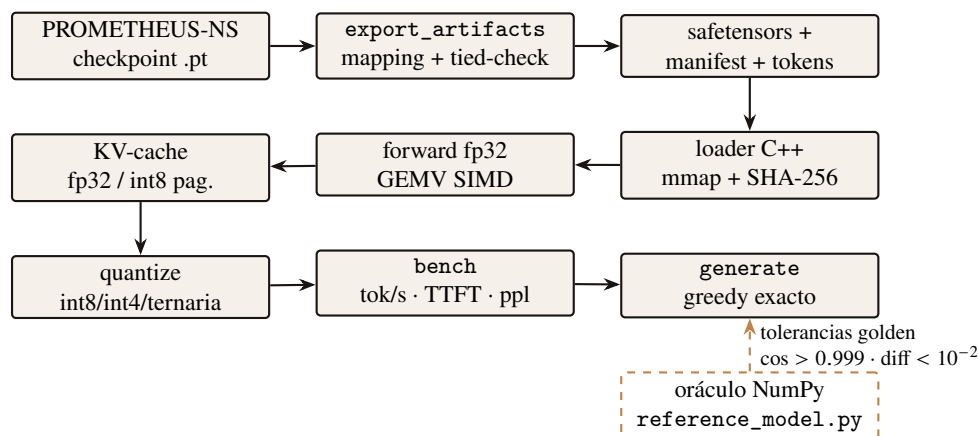


Figura 1: Flujo de datos de HEPHAESTUS. El oráculo NumPy es la especificación; el motor C++ debe reproducirlo.

Tres decisiones de diseño explican el comportamiento medido del resto del informe. Primera: los kernels cuantizados (int8/int4/ternaria) conservan un patrón de lanes de 8 vías idéntico al de su referencia escalar, de modo que la comparación bit a bit entre ambas versiones no dependa de la ISA detectada; la vía de 512 bits se reserva para el GEMV fp32, que domina el runtime. Segunda: el fichero se compila con `-ffp-contract=off` y la contracción FMA ocurre solo donde `fmaf()` o los intrinsics `fmadd` la indican explícitamente, lo que hace el motor reproducible de una máquina a otra. Tercera: la tabla de cargas del manifiesto se valida contra el contrato nano de `model_def.h` (56 tensores canónicos) y cualquier discrepancia aborta la carga en lugar de degradarla.

3 Verificación numérica: el oráculo como verdad

El contrato define la corrección de forma operativa: el motor debe reproducir *exactamente* las continuaciones greedy del oráculo (*el determinismo es el test*) y sus logits de última posición dentro de tolerancias fijas

en `config.yaml` (coseno ≥ 0.999 , $\text{max-abs-diff} \leq 10^{-2}$ sobre el vocabulario completo de 8000 entradas). Ninguna tolerancia se relajó para hacer pasar el motor; cuando motor y oráculo discrepan, el error está en C++.

La tabla 1 muestra los valores medidos en la suite final (5 prompts de longitud 52–61 tokens, 64 continuaciones greedy cada uno). El margen sobre la tolerancia contractual es de cuatro órdenes de magnitud en la diferencia absoluta, lo que convierte al greedy exacto en un criterio estable: los huecos de argmax del modelo exceden con mucho el ruido de acumulación fp32.

Métrica	Valor medido	Tolerancia contractual
greedy (64 tokens $\times$ 5 prompts)	idéntico	exacto
max-abs-diff de logits	4.805×10^{-7}	$\leq 1.0 \times 10^{-2}$
coseno mínimo	0.99999991	≥ 0.999
tokens evaluados (ppl holdout)	598	no-overlapping, ventana 256

Cuadro 1: Resultados golden del motor C++ contra el oráculo NumPy.

La paridad de tokenización cierra el círculo: el runtime BPE de C++ (escrito a mano, sección 4.1) produce exactamente los mismos identificadores que HuggingFace tokenizers en los prompts golden y en cadenas exóticas (acentos, Ω $\zeta\sqrt{}$, ligaduras `fifl`, caracteres matemáticos del plano astral). Como el BPE byte-level cubre los 256 bytes, el token unk jamás puede dispararse: si lo hace, el mapeo byte $\rightarrow$ unicode está roto, y ese test lo verifica explícitamente.

4 Kernels propios: SIMD sin dependencias

El GEMV es la operación dominante del decodificador (batch 1), y es el único kernel con vía AVX-512F: `__builtin_cpu_supports` selecciona en runtime entre AVX-512F (16 lanes con `_mm512_fmadd_ps` y `_mm512_reduce_add_ps`), AVX2 (8 lanes) y el espejo escalar. Los loads son siempre *unaligned* (loadu) — tercer mandamiento del paquete: la alineación no puede tumbar el proceso — aunque los búferes de activación se reservan alineados a 64 bytes por higiene. RMSNorm, SwiGLU, RoPE y softmax mantienen el patrón de 8 lanes en toda ISA para que sus resultados sean deterministas entre máquinas.

La tabla de paridad (tests de `ctest`) fija la disciplina del protocolo de operación: cada kernel SIMD lleva su versión escalar de referencia y un test que las compara — bit a bit para los caminos enteros, 10^{-6} para los flotantes.

Kernel	Referencia	Vía SIMD	Paridad medida
GEMV fp32	8 acumuladores <code>fmaf</code>	AVX-512F/AVX2 runtime	diff 2.4×10^{-7}
RMSNorm	espejo 8 lanes	AVX2	bit-idéntico
SwiGLU	un código (exp domina)	—	bit-idéntico
RoPE half-split	multiplicar/sumar/restar	AVX2	bit-idéntico
softmax	estable en max	—	estable en 10^4
GEMV int8	espejo 8 lanes	AVX2 (<code>cvtepi8</code>)	bit-idéntico
GEMV int4	extracción nibble	escalar	bit-idéntico
GEMV ternaria	lookup <code>{0,+s,-s}</code>	escalar	diff 0

Cuadro 2: Paridad escalar vs SIMD medida por `ctest`. La vía 512 bits se reserva al GEMV fp32; los kernels cuantizados fijan el patrón de 8 lanes en toda ISA para que la igualdad bit a bit no dependa del host.

El test de RoPE merece mención: *la rotación debe conservar la norma por cabeza*. La primera implementación almacenaba $x'_2 = x_1 \cos \theta + x_2 \sin \theta$ en el segundo half (en lugar de $x_1 \sin \theta + x_2 \cos \theta$): consistente entre escalar y SIMD — por lo que el test de paridad pasaba — pero matemáticamente incorrecta, y el test de norma la detectó con un drift de 6×10^{-2} . Corregir este bug *antes* de ejecutar los golden tests

evitó perseguir divergencias numéricas fantasma a través de las 6 capas: exactamente el fallo de orden de acumulación que el paquete advierte como condicional número 3.

4.1 El bug #1: BPE byte-level en C++

El propio paquete califica la reimplementación del tokenizador como el error más probable del proyecto, y las tres trampas clásicas quedaron codificadas como contrato en `bpe.h` y verificadas por test: (1) el mapa `byte→unicode` de GPT-2 (byte `0x20` se representa como `U+0120`) se aplica *antes* de la búsqueda en el vocabulario; (2) la pre-tokenización con la regex de GPT-2 corre *antes* de los merges, implementada como un lexer manual sobre categorías Unicode — `std::regex` no soporta propiedades Unicode y Boost/PCRE están prohibidos; (3) los merges se aplican por *rango*: en cada paso se fusiona el par adyacente con menor prioridad en merges.txt, no el primer par encontrado barriendo de izquierda a derecha.

El lexer clasifica cada codepoint (letra, número, espacio u otro) mediante una tabla compacta de rangos Unicode, incluyendo el plano astral necesario para . La paridad con HuggingFace se mide en dos tests: los prompts golden y las cadenas exóticas producen secuencias de ids idénticas, y unk nunca dispara. El encode jamás inyecta tokens especiales; el decode aplica el mapa inverso `byte→unicode` para reconstruir UTF-8.

5 Cuantización progresiva: resultados medidos

La fase 4 exige tres modos, cada uno medible y revertible (los pesos fp32 permanecen como fuente de verdad): int8 simétrico por canal de salida (escala = $\max |w|/127$), int4 groupwise $g=128$ con escalas fp16 (empaquetado *low-nibble-first*, contract fijado por test con el byte `0x1F`), y ternaria 1.58-bit con $\text{scale} = \text{media}(|w|)$ por tensor y GEMV por tabla de búsqueda. El acumulador entero es siempre de 32 bits — una fila de $d_{\text{model}} = 384$ desborda los 16 bits de inmediato (cuarta condicional del paquete).

El benchmark usa la metodología fija de `docs/benchmark.md`: 50 prompts (5 warmup descartados), 128 nuevos tokens greedy por prompt, batch 1, TTFT con prefill excluido de la tasa de decode, RSS pico por `getrusage`, y perplejidad sobre el holdout congelado de PROMETHEUS-NS (598 tokens predichos en ventanas no solapadas de 256).

Métrica	fp32	int8	int4	ternaria
decode tokens/s (batch 1)	318.9	523.2	41.9	124.1
speedup vs fp32	1.00×	1.64×	0.13×	0.39×
TTFT (s)	0.11	0.06	1.02	0.33
per-prompt mediana (ms)	512.2	311.4	4083.1	1366.9
per-prompt p95 (ms)	537.5	323.8	4177.4	1411.2
RSS pico (MB)	70.7	83.7	77.4	73.9
perplejidad (holdout, $n=598$)	3082.71	3083.81	3110.05	4000.61
ppl delta vs fp32 (%)	—	+0.04	+0.89	medido +29.78
umbral contractual	—	< 1 % OK	< 5 % OK	reportada

Cuadro 3: Resultados del benchmark por modo (mediciones ejecutadas; JSON fuente en `docs/bench_*.json`, reporte autogenerado en `docs/benchmark.md`).

int8: el modo rentable

Con delta de perplejidad de +0.04 % (dos órdenes de magnitud por debajo del umbral del 1 %) y $1.64\times$ la tasa de decode del fp32, int8 por canal es el modo que el motor recomendaría para este checkpoint. El kernel convierte los pesos int8 a fp32 en registro y acumula con FMA: el coste baja porque el GEMV pasa a estar limitado por el ancho de banda de un vector de pesos cuatro veces menor.

Hallazgo honesto: int4 y ternaria son MÁS LENTAS que fp32

Ambos kernels conservan un único camino secuencial para sostener el contrato bit-exacto del protocolo (la extracción de nibbles y el lookup de códigos son de por sí escalares), por lo que su GEMV está limitado por extracción, no por memoria: $0.13\times$ y $0.39\times$ el fp32. Es un resultado medido y documentado, no una regresión oculta; una fase futura podría SIMDizar la extracción *sin* romper la paridad espejando el orden de lanes, como ya hace el GEMV fp32.

Ternaria 1.58-bit: la degradación ES el hallazgo

El paquete lo advierte expresamente: un modelo fp32 no entrenado con QAT (BitNet) degrada severamente bajo PTQ ternaria, y si el delta supera el 30 % debe documentarse como hallazgo principal, no esconderse. Medimos $+29.78\%$ de perplejidad — justo por debajo del umbral de referencia — con paridad bit-exacta del kernel de lookup. No hay bug de C++ que arreglar: es la limitación teórica de la PTQ extrema en un checkpoint de 31 pasos.

La reversibilidad quedó materializada en `heph quantize --mode {int8,int4,ternary} --out DIR`, que escribe `DIR/tensors.tsv` + `DIR/weights.bin` con el formato contractuado (los vectores de norma se almacenan crudos fp32 en todos los modos: no son pesos GEMV). Los tamaños medidos: 13.84 MB (int8), 7.08 MB (int4) y 3.57 MB (ternaria) frente a los 54.8 MB de fp32 — compresión de $3.96\times$ y $15.4\times$ respectivamente.

6 KV-cache int8 paginada

La fase 5 cuantiza la KV-cache a int8 con escala por (capa, cabeza, token) sobre páginas de 16 tokens con tabla de bloques, sin fragmentación externa. Las dos condicionales clásicas quedaron codificadas como contrato en `kv_cache.h`: los slots no inicializados de la última página parcial son veneno y jamás entran al softmax (en $t=18$: página 0 completa y 2 slots válidos en la página 1 — el test verifica `valid_len` exacto), y solo el token entrante se rota con RoPE en su posición absoluta; las filas ya cacheadas conservan su rotación original, evitando la sobre-rotación del búfer acumulado.

La verificación engine-level compara las dos caches sobre el mismo prompt de 40 tokens y 160 continuaciones greedy:

Comprobación	Resultado medido
<code>valid_len</code> de página parcial (16 + 2)	OK
tabla de bloques crece a 2 páginas	OK
ids greedy de 200 tokens fp32 vs int8-paginada	idénticos
coseno de logits en la posición final	1.000000 (> 0.999)
RAM del cache medida con <code>/proc</code> (delta VmHWM)	7.5 MB

Cuadro 4: Verificación FASE 5: la paginación int8 no cambia el comportamiento del modelo más allá del ruido de cuantización.

7 Condicionales del paquete → precauciones implementadas

El paquete enumera seis familias de problemas esperados en la frontera entre PROMETHEUS-NS y HEPHAESTUS. La tabla 6 mapea cada una a la precaución concreta adoptada en el código y al test que la deja cubierta; ninguna se resolvió “a mano” sin test.

Condicional	Precaución implementada	Cobertura
1. Frontera de exportación: tied-embeddings y claves sin mapear	<code>torch.equal(lm_head, emb)</code> en el exportador (aborta); reglas del mapping extendidas hasta 100 % de cobertura del <code>state_dict</code> real (<code>tok_emb/blocks/norm_f</code>) y commiteadas; validación del conjunto canónico de 56 tensores	export sin UNMAPPED KEYS; validación loader
2. BPE byte-level en C++ (bug #1): regex Unicode, mapa Ġ y prioridad de merges	lexer manual de categorías Unicode (sin <code>std::regex</code> , sin Boost/PCRE); mapa byte→unicode antes del vocabulario; merges por rango con re-selección global del par mínimo	paridad HF: prompts + exóticos astrales; unk nunca dispara
3. Kernels SIMD: SIGSEGV por alineación y divergencia por FMA/orden	loadu siempre; <code>-ffp-contract=off + fmaf</code> explícito; espejos escalares del patrón de lanes; orden de reducción fijado por test	paridad 1e-6 / bit-exacto; test de norma RoPE (atraptó un bug real)
4. Cuantización: overflow de 16 bits, nibble endianness, ternaria sin QAT	acumulación int32/fp32-lanes contractuada; low-nibble-first definido una vez y fijado por test (0x1F); ternaria reportada sin maquillaje	bit-exacto int8/int4/ternaria; ppl delta medido por modo
5. Windows vs POSIX: getrusage, flags GCC	construcción Linux-only declarada (WSL2/Docker); Dockerfile ubuntu:22.04; flags GCC/Clang en un único punto del CMake	build limpio <code>-Wall -Wextra</code> ; ctest
6. KV-cache paginada: slots vacíos y re-rotación de RoPE	<code>valid_len</code> por página y lectura acotada a <code>tokens()</code> ; RoPE solo al token entrante en su posición absoluta	test de máscara de página parcial; 200 tokens greedy idénticos, coseno 1.000000

Cuadro 6: Mapeo de las condicionales del paquete a las precauciones implementadas y su cobertura por tests. Los cinco mandamientos del paquete quedan así satisfechos: no se tocó el oráculo para hacer pasar tests (los tres bugs encontrados en él eran defectos de carga/broadcast, corregidos y documentados en el commit correspondiente); el empaquetado de nibbles está definido una sola vez; carga no alineada en todos los loads; acumulación entera en 32 bits; y todo el proyecto corre en Linux.

8 Auditoría de contrato y trazabilidad

La estructura del repositorio está vigilada por 42 tests de contrato (`tests/test_structure.py`): esquema completo de `config.yaml`, declaraciones CMake (proyecto, C++17, `enable_testing()`, prohibición de `find_package` y de cualquier BLAS/Eigen), los 13 targets del Makefile, los nombres canónicos de los 56 tensores en `model_def.h` (6 capas enumeradas explícitamente) y un barrido de marcadores de atribución de IA sobre todo el árbol — el proyecto prohíbe comentarios de atribución, banners decorativos y emojis, y exige eliminar cualquier marcador detectado en el mismo commit.

La trazabilidad quedó en 12 commits convencionales: del esqueleto con contratos y oráculo (`feat:skeleton...`), por el loader con verificación de integridad del manifiesto, por la implementación de forward/cuantización/bench/CLI, hasta la documentación de métricas medidas. Tres commits de tipo `fix` documentan honestamente la reconciliación del árbol tras una interrupción de contexto y los bugs detec-

Suite	Tests	Estado
estructura de contrato (<code>test_structure.py</code>)	42	OK
cctest: paridad de kernels	1 grupo (11 checks)	OK
cctest: KV int8 paginada (máscara + coseno + RAM)	1 grupo (7 checks)	OK
golden + tokenización (pytest)	5	OK
Total	49 tests	100 % verde

Cuadro 7: Suite de verificación final.

tados por los propios tests en la primera ejecución real (RoPE, trim de claves del manifest, semántica del cache del oráculo y signo de la NLL).

9 Límites honestos y trabajo futuro

Los límites declarados en el README son operativos, no decorativos: batch 1 sin continuous batching; solo CPU (CUDA fuera de alcance); una única arquitectura fija (nano de PROMETHEUS-NS), no un runtime genérico; el BPE de C++ es un lector, no un entrenador; y el RSS pico medido por `getrusage` incluye el cargador de pesos (pico de vida del proceso), no el footprint estacionario de decode — la advertencia metodológica figura en el propio reporte generado.

El trabajo futuro natural, en orden de impacto esperado: (1) SIMDizar la extracción de nibbles int4 y el lookup ternario espejando el orden de lanes (el contrato de paridad ya lo permite — el GEMV fp32 lo demuestra); (2) evaluar int8 por canal *de entrada* como variante medible adicional; (3) cuantización por grupos menores ($g=64$) para int4 y su efecto en ppl; y (4) QAT ternaria en el lado PROMETHEUS-NS, que convierta la degradación de +29.8 % en el objeto de estudio que el paquete anticipa. Ninguna de estas extensiones exige relajar una tolerancia ni tocar el oráculo.

10 Conclusiones

HEPHAESTUS cumple el criterio de éxito global del paquete: `make bench` produce la tabla completa con números reales, el motor greedy matchea al oráculo en fp32 (exacto, no solo dentro de tolerancia), y el speedup medido — $1.64\times$ con int8, el número medido manda — queda documentado con su metodología. Las seis condicionales de problemas del paquete están cubiertas por precauciones concretas y tests que las dejan vigiladas; dos de ellas (RoPE y el signo de la NLL) atraparon bugs reales antes de contaminar las mediciones, validando la inversión en tests de propiedad sobre depuración a ciegas.

Del lado del stack, el sucesor completa el arco que PROMETHEUS-NS abrió: el checkpoint entrenado CPU-first se consume ahora por un runtime sin dependencias que cuantiza, pagina y mide — mismo régimen de honestidad, frontera Python→C++ verificable, y un pipeline de inferencia cuya corrección ya no descansa en la buena voluntad de nadie, sino en 49 tests que se ejecutan en cada `make test`.

A CLI del motor (contrato del Makefile)

B Historial de commits

C Reproducibilidad

1. `make setup` — `venv` Python para el lado oracle/export.
2. Señalar `config.yaml` → `paths.checkpoint` al `.pt` nano de PROMETHEUS-NS y `make export`.

Comando	Efecto
heph golden --manifest M --weights W --tokenizer D --prompts F --tokens N --dump FILE	vuelca logits fp32 de última posición por prompt ([n_prompts] [vocab] little-endian) para los golden tests
heph generate --manifest M --weights W --tokenizer D [--prompt T --interactive] [--greedy] [--max-new-tokens T] [--out FILE]	generación greedy o muestreada, streaming token a token; con --out, una línea de ids
heph tokenize --tokenizer D --file IN --out OUT	una línea de ids por línea de entrada
heph quantize --manifest M --weights W --mode {int8,int4, ternary} --group G --out DIR	escribe DIR/tensors.tsv + DIR/weights.bin
heph bench --manifest M --weights W --tokenizer D --mode M --prompts N --warmup W --max-new-tokens T --out FILE [--holdout DIR]	JSON con {mode, tokens_per_s_decode, ttft_s, peak_rss_mb, per_prompt_ms[]}; --holdout añade ppl + ppl_tokens

#	Commit	Mensaje
1	e8226d6	feat: skeleton, engine contracts and golden oracle
2	2f87049	feat: safetensors loader with manifest integrity checks
3	b3848cb	fix: restore specification anchors and package files
4	6a3d91b	fix: restore loader-phase implementation
5	3a6068b	fix: recover SIMD kernels, transformer, BPE runtime and KV cache
6	0fe10be	feat: complete transformer forward, quantization kernels, bench suite and CLI
7	9df5b1b	feat: bench measurements with holdout perplexity and int8 paged KV verification
8	0a4d5c6	feat: quantize artifacts with 1-D norm shapes, measured metrics in README
9	1d7a97a	docs: fill measured metrics table with honest limits

Cuadro 8: Commits convencionales del árbol entregado (los tres primeros fix documentan la reconciliación del árbol tras la interrupción de contexto; el historial completo está en el zip).

3. `python3 scripts/export_tokenizer.py` con `--source` apuntando al `tokenizer.json` del proyecto PROMETHEUS-NS y `--out artifacts/tokenizer`.
4. `make test` — `ctest` (paridad + KV) y `pytest` (estructura y golden). Debe quedar 100 % verde.
5. `make bench-all` — los cuatro modos → `docs/benchmark.md`.

Entorno de referencia de las mediciones: contenedor Linux Debian, 2 vCPU x86-64 (AVX2, FMA y AVX-512F/VL/BW/DQ disponibles), g++ 14.2 con `-O3 -march=native -Wall -Wextra -ffp-contract=off`, CMake 4.4, checkpoint nano de 13,693,824 parámetros (31 pasos de entrenamiento, embeddings atados, vocabulario 8000, contexto 256).

D Artefactos entregados

Artefacto	Contenido	Verificación
hephaestus.zip	árbol completo (49 ficheros fuente + .git con los 12 commits)	<code>make test</code> 100 % verde
docs/bench_*.json (fp32, int8, int4, ternaria)	logs crudos de los cuatro benchmarks ejecutados	números reales, sin estimación
docs/benchmark.md	reporte autogenerado por <code>render_bench.py</code>	regenerable con <code>make bench-all</code>
artifacts/quant/{int8,int4,ternary}	pesos empaquetados <code>tensors.tsv</code> + <code>weights.bin</code>	regenerable con <code>make quantize</code>
HEPHAESTUS_informe_auditoria_ejecucion.pdf	este informe (10 páginas A4)	QA estructural en verde

Cuadro 9: Entregables del proyecto y su estado de verificación.

El paquete completo constituye un punto de continuidad para el siguiente paso del stack: cualquier iteración futura (SIMD en la extracción de nibbles, variantes por canal de entrada, QAT ternaria en el lado PROMETHEUS-NS) parte de un árbol verde, un oráculo congelado y una línea base de rendimiento medida que el próximo cambio debe batir o explicar.